

✓ Title:- Design RNN or its variant including LSTM or GRU

- Select a suitable time series dataset. E.g - Predict sentiments based on product reviews.
- Apply for prediction

✓ Import Required Libraries

```
import numpy as np
import tensorflow as tf
from tensorflow.keras.datasets import imdb
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, LSTM, Dense, Dropout
from tensorflow.keras.callbacks import EarlyStopping
```

✓ Load Dataset

```
vocab_size = 10000
(X_train, y_train), (X_test, y_test) = imdb.load_data(num_words=vocab_size)

print("Training samples:", len(X_train))
print("Testing samples:", len(X_test))
```

Downloading data from <https://storage.googleapis.com/tensorflow/tf-keras-datasets/imdb.npz>
17464789/17464789 ————— 0s 0us/step
 Training samples: 25000
 Testing samples: 25000

✓ Preprocessing (Padding Sequences)

```
max_len = 200

X_train = pad_sequences(X_train, maxlen=max_len)
X_test = pad_sequences(X_test, maxlen=max_len)

print(X_train.shape)

(25000, 200)
```

✓ Build LSTM Model

```
model = Sequential()
model.add(Embedding(input_dim=vocab_size, output_dim=128, input_length=max_len))
model.add(LSTM(128, return_sequences=False))
model.add(Dropout(0.5))
model.add(Dense(1, activation='sigmoid'))

model.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])

model.summary()
```

/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/embedding.py:100: UserWarning: Argument `input\_length` is deprecated
 warnings.warn(

Model: "sequential"

Layer (type)	Output Shape	Param #
embedding (Embedding)	?	0 (unbuilt)
lstm (LSTM)	?	0 (unbuilt)
dropout (Dropout)	?	0
dense (Dense)	?	0 (unbuilt)

Total params: 0 (0.00 B)

Trainable params: 0 (0.00 B)

Train Model

```
early_stop = EarlyStopping(monitor='val_loss', patience=2)

history = model.fit(
    X_train, y_train,
    epochs=5,
    batch_size=64,
    validation_split=0.2,
    callbacks=[early_stop]
)
```

```
Epoch 1/5
313/313 ————— 175s 546ms/step - accuracy: 0.7933 - loss: 0.4395 - val_accuracy: 0.8576 - val_loss: 0.3347
Epoch 2/5
313/313 ————— 194s 523ms/step - accuracy: 0.9022 - loss: 0.2538 - val_accuracy: 0.8664 - val_loss: 0.3236
Epoch 3/5
313/313 ————— 164s 525ms/step - accuracy: 0.9265 - loss: 0.1934 - val_accuracy: 0.8662 - val_loss: 0.3726
Epoch 4/5
313/313 ————— 162s 518ms/step - accuracy: 0.9452 - loss: 0.1505 - val_accuracy: 0.8670 - val_loss: 0.3755
```

Evaluate Model

```
loss, accuracy = model.evaluate(X_test, y_test)
print("Test Accuracy:", accuracy)
```

```
782/782 ————— 84s 107ms/step - accuracy: 0.8602 - loss: 0.3849
Test Accuracy: 0.8601599931716919
```

Prediction Example

```
sample = X_test[0].reshape(1, -1)
prediction = model.predict(sample)

print("Predicted Sentiment:", "Positive" if prediction[0][0] > 0.5 else "Negative")
```

```
1/1 ————— 0s 222ms/step
Predicted Sentiment: Negative
```

Prediction on Unseen data

```
from tensorflow.keras.datasets import imdb

word_index = imdb.get_word_index()
```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/imdb\_word\_index.json
1641221/1641221 ————— 0s 0us/step
```

```
def encode_review(text):
    words = text.lower().split()
    encoded = []

    for word in words:
        if word in word_index and word_index[word] < vocab_size:
            encoded.append(word_index[word] + 3) # +3 because IMDB reserves indices
        else:
            encoded.append(2) # unknown word

    return encoded
```

```
from tensorflow.keras.preprocessing.sequence import pad_sequences

def preprocess_review(text):
    encoded = encode_review(text)
    padded = pad_sequences([encoded], maxlen=max_len)
    return padded
```

```
review = "This movie was absolutely amazing, I loved it"

processed_review = preprocess_review(review)
```

```
prediction = model.predict(processed_review)

print("Prediction Score:", prediction[0][0])

if prediction[0][0] > 0.5:
    print("Sentiment: Positive 😊")
else:
    print("Sentiment: Negative 😞")
```

1/1 ————— 0s 96ms/step
Prediction Score: 0.8587036
Sentiment: Positive 😊

```
review1 = "The movie was fantastic and full of emotions"
review2 = "Worst movie ever, waste of time"

for r in [review1, review2]:
    pred = model.predict(preprocess_review(r))
    print(r)
    print("Sentiment:", "Positive" if pred[0][0] > 0.5 else "Negative")
    print()
```

1/1 ————— 0s 67ms/step
The movie was fantastic and full of emotions
Sentiment: Positive

1/1 ————— 0s 73ms/step
Worst movie ever, waste of time
Sentiment: Negative